Cómo funciona el código

Alcance: P08, aproximación a un solo waypoint. Sin cambios de pines.

Estado instalado: ESP v27, carga y hash verificados; Nicla N03-panel-v5-wide. La última prueba física fue v26: hubo acercamiento y correcciones, choques con techo/pared y ninguna llegada. La mejora de yaw v27 aún requiere prueba física.

1. Datos: de la cámara a la ESP

Nicla: captura RGB565 a 240 × 160, aplica el filtro rojo LAB y confirma un seguimiento. Envía por UART posición, tamaño, estado y muestra. FULL = completo plausible; PARTIAL = seguimiento parcial confirmado. Un candidato con flag0 no habilita navegación.

ESP: FullBicopter::sense() usa NiclaSuite para reunir visión con IMU, barómetro y batería. SensorSuite recoge los sensores base; NiclaSuite agrega la visión. SensorMap da nombre a los índices de **app.sensors**.

2. Decisión y realimentación

main.cpp / loop(): processCommands() → sense(app.sensors) → TestModes::run(app). En P08 se ejecuta BalloonMission: SEARCH → APPROACH → VISIT_CONFIRM → DONE. Un fallo termina el ensayo.

Altura: InertialHeight combina error vertical visual y velocidad barométrica. Adapta el trim de sustentación; puede descargar empuje hasta el mínimo del ESC. Con PARTIAL por debajo del centro limita que una caída indicada sólo por presión invierta la corrección. Con movimiento visual fiable conserva anticipación.

Yaw y avance: SupportYaw calcula corrección de rumbo y mezcla los mandos. ImpulseApproach alterna alinear, acomodar, impulso, retorno y corrección durante inercia. En v27 se proyecta el giro durante el intervalo sin corrección para evitar iniciar o prolongar un pulso que descentre el objetivo.

3. Salida a los actuadores

applyPhysicalCommand() convierte ángulos a pulsos y llama a **RawBicopter::commandMotorPowerTest()**. Con actuadores armados, AServo recibe microsegundos y BLMotor::act() convierte potencia a PWM del ESC. Arranca desarmado; STOP mantiene throttle mínimo.

Lecturas con límites: el barómetro entrega una estimación relativa con deriva; el tamaño visual no mide contacto. Potencia y ángulos telemétricos son órdenes, no empuje, RPM ni posición efectiva medidos. Este perfil nuevo de P08 no acredita todavía las pistas completas P10/P11.

Qué ajustar entre pruebas

Los nombres de archivo remiten al índice de rutas completas de la página 4.

Ajuste	Dónde y valor actual	¿Recargar ESP?
Pose base de servos	SupportYaw.h: VISUAL_BASE1_DEG = 52,5°; VISUAL_BASE2_DEG = 64,5°. Se calculan con referencias48,5°/68,5° y FORE_AFT_CORRECTION_DEG = 4°.	Sí
Velocidad y avance de servos	SupportYaw.h: SERVO_RATE = 30°/s; inclinación normal hasta12°, hasta18° en la condición de descarga durante subida.	Sí
Sustentación inicial	BalloonMission.cpp: approachBase_ = 0.15 en entrada directa. VisualSearch.h inicia búsqueda en0.12; al pasar de búsqueda a aproximación conserva el trim aprendido.	Sí, para cambiar esos valores iniciales
Adaptación de sustentación	InertialHeight.h: tras error visual sostenido600ms, adapta trim con límites y antiwindup. No impone una potencia fija de hover.	Sí, para cambiar la ley de adaptación
Techo del ensayo	check_single_waypoint.py: alt_max_power=20. AdaptiveLift.h impone HARD_MAX=0.20. El mínimo de mando permitido puede llegar a0.	No para bajar el techo enviado; sí para superar el límite del firmware
Respuesta de yaw P08	BalloonMission.cpp: kp=0.35, ki=0.08, kd=3; autoridad0.25, banda de mando1°. SupportYaw limita la velocidad deseada a0.20rad/s.	Sí
Impulsos y tolerancias	ImpulseApproach.h: acomodar500ms, pulso máximo1200ms, inercia2000ms. BalloonMission.cpp: entrada±24px en x y ±50px en y; yaw±8° y velocidad≤0.12rad/s, además de proyección v27.	Sí
Duración de prueba	Lanzador: --seconds 60. Admite5..120 segundos después de la cuenta. El archivo02_PRUEBA_WAYPOINT.cmd fija60.	No

Dos detalles que evitan confusiones

• Los ángulos52,5°/64,5° también aparecen en la preparación de check_single_waypoint.py, en el diccionario **zero**. Si se cambia la pose base, revisar ambas ubicaciones.
• Cambiar ganancias de P03 en Ground no cambia automáticamente las ganancias fijadas para P08. vis_min_power=10 del lanzador tampoco es una potencia neutra aprendida ni el mínimo efectivo del regulador vertical.

La mejora de configuración que falta

Exponer por Ground un único perfil P08 con **servo1 base, servo2 base, sustentación inicial, techo, kp, ki y kd de yaw**. Así podrían variarse entre ensayos sin recompilar. **Ese perfil de configuración aún no está implementado:** hoy sólo duración y límites ya enviados admiten cambios sin recargar firmware. Conviene variar un grupo cada vez y registrar versión, valores y resultado físico.

Cómo repetir la prueba

Windows CMD normal. No requiere consola como administrador.

1. Ground en COM9

Ejecuta este archivo y conserva su ventana abierta. Si el puerto8765 ya tiene un puente, el lanzador evita abrir otra instancia.

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\groundstation\01_GROUND.cmd
```

2. Prueba CON MOTORES, en otra ventana

Ejecuta el siguiente archivo. Verifica telemetría, batería, cámara y firmware compatible; prepara los ESC, anuncia la cuenta por voz y limita la ejecución a60s después de la cuenta.

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\groundstation\
02_PRUEBA_WAYPOINT.cmd
```

Antes del vuelo: ESP y Nicla montadas y alimentadas por la PCB; únicamente Ground conectado por USB al PC. Para cargar firmware, la placa correspondiente va por USB fuera de la PCB y sin batería. Es la restricción eléctrica específica de este montaje.

3. Qué comprobar en consola

Mensaje o campo	Qué indica
FIRMWARE_CONFIRMADO: 27	La ESP que responde ejecuta la versión nueva. El lanzador acepta26 o posterior para poder repetir la prueba anterior.
Objetivo fijado. Aproximando.	Se obtuvo seguimiento confirmado; no significa llegada.
fase / avance / frenar_giro_antes_de_impulsar	Estado del pulso, demanda horizontal y limitación por predicción de yaw.
trim / potencia / freno_vertical	Nivel de sustentación adaptado, orden de motor y motivo de corrección. No son empuje real medido.
WAYPOINT_COMPLETADO	La lógica confirmó llegada estimada. La voz anuncia el resultado al terminar.
STOP_CONFIRMADO: True	Llegó telemetría nueva con modo0 y armed0 después de ordenar STOP.

4. Interrumpir, guardar y evaluar

Usa STOP de la app o Ctrl+C en la ventana de prueba y comprueba **STOP_CONFIRMADO: True**. Los JSONL de cada prueba quedan en la carpeta de abajo; Ground también guarda CSV generales en logs. No cierres Ground durante la ejecución.

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\logs\single_waypoint
```

Llegada actual: ancho visual≥220px, centrado±24px en ambos ejes, al menos un FULL previo, tres muestras nuevas y persistencia≥500ms. Es una calibración provisional, no un sensor de contacto. La última prueba llegó a154px y no tocó el waypoint: no confirmó llegada.

Índice de rutas completas

Proyecto utilizado: carpeta Competencia. Las rutas se pueden seleccionar y copiar.

A · Bucle principal / setup(), loop()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\main.cpp
```

B · Recolección del robot / FullBicopter::sense()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\lib\BlimpSwarm\src\
robot\FullBicopter.cpp
```

C · IMU, barómetro y batería / SensorSuite::update()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\lib\BlimpSwarm\src\
sense\SensorSuite.cpp
```

D · Unión de sensores y visión UART / NiclaSuite

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\lib\BlimpSwarm\src\
sense\NiclaSuite.cpp
```

E · Nombres de los índices de sensores

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\app\SensorMap.h
```

F · Producción Nicla; se copia como main.py a su unidad USB

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\vision\n03_panel\
nicla_balloon_mission_main.py
```

G · Misión P08 / updateN03Search(), updateN03Approach()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\mission\
BalloonMission.cpp
```

H · Ángulos y mezcla / calibratedYaw(), command(), Mixer::update()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\app\SupportYaw.h
```

I · Sustentación adaptativa / InertialHeight::update()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\mission\
InertialHeight.h
```

J · Pulsos y proyección de rumbo / headingFits(), update()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\mission\
ImpulseApproach.h
```

K · Ejecución PWM / commandMotorPowerTest()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\lib\BlimpSwarm\src\
robot\RawBicopter.cpp
```

L · Conversión de potencia a PWM de ESC / BLMotor::act()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\lib\BlimpSwarm\src\act\
BLMotor.cpp
```

M · Lanzador y registro de ensayo / WaypointTrial.run()

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\groundstation\
check_single_waypoint.py
```

N · Puente COM9, ESP-NOW y WebSocket

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\groundstation\
telemetry_bridge.py
```

O · Límites y aprendizaje de trim

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\firmware\src\mission\
AdaptiveLift.h
```

P · Reportes de cambios y esta guía

```
C:\Users\stefa\Desktop\BLIMP\Competencia\Blimp\Blimp\Codigos-del-hardware\docs
```